



AUDITORIA DE ARQUITETURA & PLANO DE DESENVOLVIMENTO

Arquitetura de autonomia governável

Do gerador semi-automático ao agente que decide, cria com qualidade real, publica sozinho e aprende o padrão do cliente — com você no controle do quanto de autonomia ligar.

DOCUMENTO

Auditoria v2 + Plano de Dev

MÉTODO

Leitura do código (âncoras de linha)

DATA

Junho de 2026

BASE

commit 705594b

O que você vai encontrar

1 · Sumário executivo — a visão e o que ela exige	03
2 · O que da plataforma atual serve (e o que não)	04
3 · A cadeia autônoma — o elo que falta	06
4 · Pilar I — Criativo de qualidade real (híbrido)	07
5 · Pilar II — Autonomia governável (painel + robô)	09
6 · Pilar III — Aprendizado configurável do "bom post"	11
7 · Plano de desenvolvimento em fases	13
8 · Manutenção, escala, performance e disponibilidade	15
9 · Conclusão e recomendação de execução	17

01 Sumário executivo — a visão e o que ela exige

A meta não é "ligar a publicação automática". É transformar a plataforma num **sistema autônomo orientado por agente**: um robô que, no horário configurado, decide o que postar com base no que o cliente quer, gera um criativo de **qualidade real** (não imagem genérica), publica sozinho quando autorizado, e **aprende** o que funciona para aquele cliente — tudo sob um **painel de governança** onde você escolhe o grau de autonomia. Esta auditoria mede o que da base atual serve para isso e desenha o caminho.

~60%

DA BASE DE AUTONOMIA

já existe como serviços reutilizáveis

3

PILARES A CONSTRUIR

criativo · autonomia · aprendizado

4

PEÇAS NOVAS CENTRAIS

+ 1 painel, 2 entidades, 1 job

A descoberta que muda o investimento

A máquina de publicação já é autônoma por baixo — agendar, publicar real (Story/Feed/Carrossel), coletar métricas, respeitar teto de gasto e kill-switch **já funcionam**. O que falta não é construir o motor, é **amarrar os elos numa cadeia** e dar a você o **volante**. Em concreto, três frentes:

- **Criativo**: um "diretor de arte" de IA que decide, por pauta, entre foto generativa, banco de imagens ou composição gráfica — e providers de imagem reais plugados.
- **Autonomia**: um painel *Configurações > Automação* com feature-flags e horários, e um *job de background* (o robô) que dispara a cadeia sozinho.
- **Aprendizado**: uma tela onde o operador escolhe o que é "bom post" (likes, saves, alcance...), salvo no banco, alimentando o que a IA prioriza.

Tese de uma linha: não se constrói um agente autônomo do zero — *completa-se e governa-se* um que já tem o motor pronto. Isso muda custo, risco e prazo a favor do projeto. O trabalho é arquitetural e cirúrgico, não um recomeço.

02 O que da plataforma atual serve (e o que não)

Filtro honesto do código atual sob a lente da nova visão. Verde = aproveita como está; âmbar = existe mas precisa estender; vermelho = falta construir.

● Aproveita ● Estende ● Constrói

CAPACIDADE SOB A NOVA VISÃO	VEREDITO	EVIDÊNCIA NO CÓDIGO / O QUE MUDA
Publicação real Story / Feed / Carrossel	Aproveita	<code>worker/Publishing/Publishers.cs</code> — os 3 formatos publicam de verdade.
Agendar → publicar → coletar métricas (jobs 24/7)	Aproveita	6 jobs no worker; <code>PublishSchedulerJob</code> (60s) + <code>PublishJob</code> (30s) já automáticos.
Teto de gasto + kill-switch + auditoria	Aproveita	<code>Budget.MonthlyCapUsd</code> , <code>Loop:Enabled</code> , <code>AuditService.LogAsync</code> .
Geração assíncrona (6 agentes) reutilizável por API	Aproveita	<code>ContentController.GenerateAsyncStart:88</code> — chamável por um orquestrador.
Coleta de métricas reais do Instagram	Aproveita	<code>MetricsCollectorJob.cs:35-122</code> — reach, likes, saves reais.
Config por workspace (fuso, janela, aprovação)	Estende	<code>Workspace.PublishWindowStart/End</code> existe, mas é aviso suave; falta virar regra.
Loop autônomo	Estende	<code>AutonomousLoopJob.cs</code> — gera <i>ideia</i> e para ; não leva até publicar.
Aprovação automática (modo já existe)	Estende	<code>ApprovalMode</code> (<code>Enums.cs:7</code>) existe; falta o gatilho que auto-aprova.
Geração de imagem de qualidade real	Constrói	Caminho padrão é determinístico/gradiente; providers reais existem mas não no fluxo padrão.
Decisão de estratégia de criativo (foto/banco/gráfico)	Constrói	Pipeline é caminho único; não há "diretor de arte" que escolha.
Orquestrador autônomo (o "robô" por horário)	Constrói	Não existe job que dispare a cadeia completa por agendamento.
Painel de governança (flags + horários + estratégia)	Constrói	Não existe a tela <i>Configurações > Automação</i> .
"Bom post" configurável (pesos de métrica)	Constrói	<code>PerformanceAnalyzer.cs:167</code> — hoje <code>Likes+Saves</code> hardcoded.

Leitura: seis capacidades verdes sustentam o projeto; as vermelhas são pontos de extensão limpos sobre código bem fatorado (feature-folders, providers abstratos, jobs isolados). Nada exige reescrever o núcleo.

Os 8 pontos da lista inicial (logo, referências, fundo, CTA/subtítulo, editar/lote/anti-colisão de agenda, dedup, lookahead) continuam válidos e estão absorvidos no plano — o logo já está pronto; os demais

entram nas Fases 1–2 como base do criativo e da esteira de agendamento.

03 A cadeia autônoma — o elo que falta

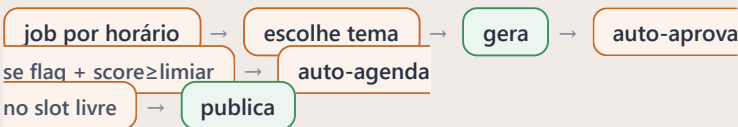
Hoje a cadeia tem as duas pontas automáticas (gerar e publicar) mas **quebra no meio**: o loop cria uma ideia e para, esperando um humano promover, gerar, aprovar e agendar. O trabalho é fechar o circuito com um orquestrador e salvaguardas.

Como é hoje



O loop autônomo (`AutonomousLoopJob.cs:100-109`) chega ao "cria ideia" e para. Tudo após depende de cliques humanos.

Como precisa ser (autonomia governável)



Verde = serviço já existe e é reutilizável. Âmbar = peça nova. As novas são poucas e bem delimitadas.

PASSO DA CADEIA	ESTADO	SERVIÇO REUTILIZÁVEL / O QUE FALTA
Trigger por horário configurado	Construir	Novo <code>PostingScheduleJob</code> (lê flags + horários do workspace).
Escolher tema/pauta autônomo	Construir	Lógica de seleção (histórico, objetivos, anti-repetição) — hoje a ideia é genérica.
Gerar conteúdo	Existe	<code>ContentController.GenerateAsyncStart:88</code> + <code>AgentsClient</code> .
Auto-aprovar (com gate de qualidade)	Estender	<code>ApprovalController.Decide:64</code> existe; falta o gatilho automático sob flag.
Auto-agendar no slot livre	Estender	<code>ScheduleController.Schedule:38</code> existe; falta escolher a hora pela janela.
Publicar	Existe	<code>PublishSchedulerJob</code> + <code>PublishJob</code> — automáticos.
Auditar cada passo	Existe	<code>AuditService.LogAsync:27</code> — registrar o que o robô fez.

04 Criativo de qualidade real (estratégia híbrida)

O ponto que você mais enfatizou: **nada de imagem genérica**. Um agente deve decidir, por pauta, a melhor estratégia de criativo e produzir algo que reflita o contexto real do cliente.



Diretor de Arte de IA

NOVO AGENTE · DECIDE A ESTRATÉGIA

Um **Creative Director Agent** entra entre o arquiteto de história e o compositor visual **NOVO**. Recebe a pauta (tema, tipo de slide, emoção-alvo) e o contexto da marca (paleta, tom, referências) e decide:

tema "lançamento" + destaque → foto generativa

tema "depoimento" + confiança → banco de imagens

tema "comparativo" + dados → composição gráfica

Ponto de inserção: `agents/src/agents/pipeline-v2.ts:37-44`. O design-spec já entrega paleta + mood — o agente só precisa consumir e rotear.



Providers de imagem reais

ESTENDER · QUALIDADE DE FOTO

A abstração `IImageProvider` já existe e já tem Gemini e OpenAI `gpt-image` implementados de verdade **REUSA**. Plugar novos é uma classe + um caso no seletor:

- **Flux / Replicate** para foto generativa de alto padrão, com a paleta da marca injetada no prompt (brand-locking de cor) **NOVO**.
- **Banco de imagens** (ex.: Unsplash/Pexels) como provider de "stock" curado por query temática **NOVO**.
- Cada provider falha de forma *diagnosticável* — sem cair em imagem genérica silenciosa.

Âncora: `agents/src/.../imageProvider.ts:18-155` · `config.ts:125-155`.



Contexto → prompt + nota de qualidade visual

ESTENDER · FECHA O LOOP

Hoje o prompt de imagem é genérico (estilo + cor). A melhoria liga o **contexto real da pauta** ao prompt (tema, produto, mensagem) e estende o **validador de qualidade** (6º agente) para dar uma *nota visual* — coerência com a paleta/mood, detecção de resultado pobre — que reprova e tenta outra estratégia automaticamente.

Âncoras: `design-spec.ts:206-212` (estética) · `quality-validator.ts:448-461` (score). O validador hoje pesa 40% técnico + 60% voz; ganha um eixo visual.

Resultado: cada post passa a ter a estratégia visual certa para o seu tema, com a cara da marca travada (cor, logo, fonte) e uma nota de qualidade que impede peça fraca de ir ao ar. Tudo por extensão dos agentes — sem tocar no render core.

05 Autonomia governável (o painel + o robô)

Você no volante: uma tela onde se liga/desliga a autonomia, define horários e regras, e um job de background que executa sozinho conforme essa configuração.



Tela "Configurações > Automação"

NOVO · O VOLANTE

● ● ● /settings/automation

Automação da publicação

MODO DE OPERAÇÃO

● Manual (aprovo tudo) Assistido Automático (robô posta sozinho)

HORÁRIOS DE POSTAGEM

Seg, Qua, Sex 09:00 18:00 + adicionar

ESTRATÉGIA DE CRIATIVO

Híbrido (IA decide)

APROVAR AUTOMÁTICO SE A NOTA FOR ≥

85 / 100

TETO DE GASTO MENSAL

US\$ 30,00 kill-switch global sempre ativo

Reaproveita o framework de `BudgetController` e `WorkspaceSettingsController` (já têm GET/PUT). É um painel novo sobre controllers existentes + alguns campos novos.



Onde a configuração mora

ESTENDER · ENTIDADES

A config por workspace já vive em `Budget` e `Workspace`. Adicionar as novas flags é uma **migration simples** + 5 linhas no controller:

NOVA FLAG	TIPO	PARA QUÊ
<code>AutoPostEnabled</code>	bool	Liga/desliga o robô para o workspace.
<code>PostingScheduleDays/Times</code>	JSON	"Seg/Qua/Sex às 09:00 e 18:00".
<code>CreativeStrategy</code>	texto	Híbrido / sempre-foto / sempre-gráfico.
<code>AutoApprovalThreshold</code>	int	Nota mínima para auto-aprovar.

Âncora: `Entities.cs` (`Budget :343`, `Workspace :22`) · `WorkspaceSettingsController.cs:24`.



O robô — `PostingScheduleJob`

NOVO · O MOTOR DA AUTONOMIA

Um novo job de background (tick a cada hora) que, para cada workspace com `AutoPostEnabled`: confere se chegou um horário configurado → escolhe o tema → chama a geração → aplica o gate de qualidade → auto-aprova se passar → agenda no próximo slot livre → deixa o `PublishJob` publicar. **Cada passo é auditado**; o teto de gasto e o kill-switch global continuam soberanos.

Modela-se sobre os jobs existentes (`apps/worker/Jobs/`) — mesma forma, mesmo acesso a banco; nada de infraestrutura nova.

Salvaguarda de marca. O modo "Automático" é opt-in por workspace com uma **rampa de confiança** recomendada: nas primeiras N publicações o sistema ainda pede um olhar humano; depois de um histórico bom, libera o 100% sozinho. Você pode pular a rampa se quiser controle total no dia 1 — é uma opção do painel, não uma trava no código.

06 Aprendizado configurável do "bom post"

A IA deve aprender o padrão de *cada* cliente — e o que conta como sucesso deve ser escolhido por você no painel, não fixado no código.



O que já existe

REUSA · A BASE DO LOOP

- Coleta real de **alcance, likes e saves** do Instagram, por post REUSA — `MetricsCollectorJob.cs:35-122`.
- Um analisador que agrega por **formato e horário** e injeta um resumo na próxima geração REUSA — `PerformanceAnalyzer.cs:14-79`.
- Histórico com as dimensões certas (formato, horário, métricas) já gravado nos dados.



O que falta — e o que você pediu

NOVO · "BOM POST" CONFIGURÁVEL

Hoje "bom post" é **Likes + Saves** em peso igual, fixo no código (`PerformanceAnalyzer.cs:167`). A evolução:

● ● ● /settings/automation > o que é um bom post

SELECIONE OS SINAIS QUE MAIS IMPORTAM PARA ESTA MARCA

☒ Salvamentos peso 40%

☒ Alcance peso 30%

☒ Curtidas peso 20%

☒ Comentários peso 10%

A seleção é salva numa nova entidade `MetricWeightConfig` (1 por workspace) NOVO; o analisador passa a calcular o sucesso com esses pesos, e o robô prioriza temas/formatos/horários que pontuam alto sob a régua do cliente.

Ponto de conexão: `Workspace` (que já agrupa `Budget` e templates). O cálculo hardcoded em `PerformanceAnalyzer.cs:167` passa a ler os pesos.

Efeito composto: com criativo de qualidade (Pilar I), robô autônomo (Pilar II) e régua de sucesso por cliente (Pilar III), o sistema vira um ciclo que melhora sozinho: *gera melhor* → *mede pelo que importa* → *aprende* → *escolhe melhor o próximo*.

07 Plano de desenvolvimento em fases

Ordenado por dependência e por valor entregue. Cada fase deixa o sistema utilizável ao final.

FASE 0 Fundação de input criativo

~1 semana · base para tudo · resolve os pendentes da lista inicial

Liga referências ao pipeline, adiciona fundo custom e subtítulo/CTA como entrada. É a matéria-prima que o diretor de arte e o robô vão usar. (Logo já está pronto.)

FASE 1 Pilar I — Criativo de qualidade real

~2–3 semanas · o maior salto de valor percebido

- Creative Director Agent (decide foto / banco / composição).
- Providers reais: Flux/Replicate + banco de imagens, com brand-locking de cor e logo.
- Nota de qualidade visual no validador, com nova tentativa automática se reprovar.

FASE 2 Pilar II — Autonomia governável

~2–3 semanas · depende da Fase 0/1 para gerar bem o que vai postar sozinho

- Entidades + migration das flags; painel *Configurações* › *Automação*.
- `PostingScheduleJob`: o robô que dispara a cadeia por horário.
- Auto-aprovação sob gate de qualidade + auto-agendamento no slot livre (anti-colisão no servidor).
- Esteira de agendamento robusta: editar, lote, "quantos à frente".

FASE 3 Pilar III — Aprendizado configurável + dedup

~2 semanas · fecha o ciclo de melhoria contínua

- `MetricWeightConfig` + painel de "o que é um bom post" (listbox + pesos).
- Analisador passa a usar os pesos; seleção de tema do robô prioriza o que pontua alto.
- Dedup editorial: não repetir tema recente; coletar também comentários.

FASE 4 Endurecimento — manutenção, escala, performance

~2 semanas · pode correr em paralelo conforme a tração

- Throttle de concorrência no gerador de imagem; resiliência da fila a reinício.
- Limite de taxa de geração por workspace; observabilidade do robô (painel do que ele fez).
- Preparar o ciclo de vida do runtime (.NET 8 → LTS antes do fim de suporte).

Visão consolidada

FASE	PILAR / FOCO	PEÇAS NOVAS CENTRAIS	ESFORÇO
0	Input criativo	costura de referências, fundo, subtítulo	~1 sem
1	Criativo de qualidade	Creative Director + providers reais + visual score	2–3 sem
2	Autonomia governável	painel + flags + PostingScheduleJob	2–3 sem
3	Aprendizado configurável	MetricWeightConfig + dedup	~2 sem
4	Escala & manutenção	throttle, resiliência, .NET LTS	~2 sem

Estimativas relativas, um desenvolvedor familiarizado com a base, reaproveitando a estrutura existente. Não incluem o App Review da Meta (trâmite externo da publicação real).

08 Manutenção, escala, performance e disponibilidade

Os requisitos que você pediu — fácil manutenção, alta escala, performance e disponibilidade para atualizações futuras — confrontados com o estado real.

O que já protege esses atributos

- **Manutenção:** domínio numa lib compartilhada; código por feature-folder; providers abstratos; contratos de enum verificados entre .NET e TypeScript. Adicionar um provider ou uma flag é local e previsível.
- **Disponibilidade:** healthchecks, reaper de geração órfã, idempotência de publicação, retry com backoff, segredos cifrados, modo degradado de primeira classe.
- **Escala (no porte atual):** API sem estado; um deploy por cliente isola a carga; fila no Postgres sem broker a operar.

Os três tetos a tratar antes de crescer (Fase 4)

R1 · Registro de jobs em memória + fila por polling

O registro de trabalhos dos agentes se perde em reinício (`agents/src/jobs.ts`); a varredura de agendamento a cada 60s vira gargalo com dezenas de milhares de posts. Aceitável hoje; tratar para multi-tenant de alto volume.

R2 · Gerador de imagem sem limite de concorrência

Slides rasterizam em paralelo sem teto — com o robô gerando sozinho em escala, vira risco de memória. Um limitador de concorrência resolve, barato e localizado.

R3 · Modelo de dados acoplado ao Instagram

Fila, agenda e rate-limit assumem Instagram. Correto hoje; ao abrir para outras redes (evolução futura), abstrair um "adaptador de rede" antes evita reescritas.

Custo do robô. Autonomia significa o sistema gastando IA sozinho. O teto mensal por workspace já existe e o protege; a Fase 4 adiciona um limite de taxa de geração por workspace para o caso de uso intenso, fechando a porta a surpresas de fatura.

09 Conclusão e recomendação de execução

A Social AI Platform tem o **motor de autonomia pronto** e bem arquitetado. O que falta é dar a ele **qualidade de criativo**, um **volante de governança** e uma **régua de aprendizado por cliente** — três pilares construídos por extensão, não por reescrita.

A sequência recomendada

0

Fundação criativa

Liga referências, fundo e subtítulo — a matéria-prima dos pilares.

1

Pilar I — Criativo de qualidade

Resolve a dor central ("nada de imagem genérica") e dá valor visível antes de automatizar.

2

Pilar II — Autonomia governável

O painel e o robô. Vem depois do criativo: o que se posta sozinho precisa já ser bom.

3

Pilar III — Aprendizado configurável

Fecha o ciclo: o robô passa a escolher pelo que cada cliente define como sucesso.

4

Endurecimento

Remove os tetos de escala antes que o crescimento os encontre.

O ponto central. A ordem importa: *qualidade antes de autonomia, autonomia antes de aprendizado em escala*. Automatizar a publicação de um criativo fraco multiplica o problema; ensinar o robô a escolher antes de medir certo o ensina errado. Esta sequência entrega valor a cada fase e protege a marca do cliente em cada passo.

Documento gerado por auditoria de código (base: commit 705594b), recalibrado para a visão de autonomia governável. Cada diagnóstico carrega a âncora de arquivo/linha verificada. Estimativas de esforço são relativas e assumem reaproveitamento da estrutura existente.